

# Problem I. 最近点对

## 题意

求平面上  $n$  个点之间的曼哈顿距离最近点对。

## 解题思路

- *Step1*: 将所有点按  $x$  坐标大小排序。
- *Step2*: 根据  $x$  下标大小进行分治, 使得点集分为两个集合  $S_L$  和  $S_R$ 。
  - 递归求解  $S_L$  和  $S_R$  内部的最近曼哈顿距离最近点对。
  - 求解一个点属于  $S_L$ , 另一个点属于  $S_R$  的点对的最近曼哈顿距离。

解题的重点在于如何求解一个点属于  $S_L$ , 另一个点属于  $S_R$  的点对的最近曼哈顿距离;

首先肯定是不能直接枚举求解的 (这样分治还有什么意义呢QAQ)

但可以考虑如何对枚举进行一些优化:

设  $\delta_L, \delta_R$  分表示  $S_L, S_R$  集合中的最小曼哈顿距离, 令  $\delta = \min\{\delta_L, \delta_R\}$ , 则只需要枚举以下区域的点对:

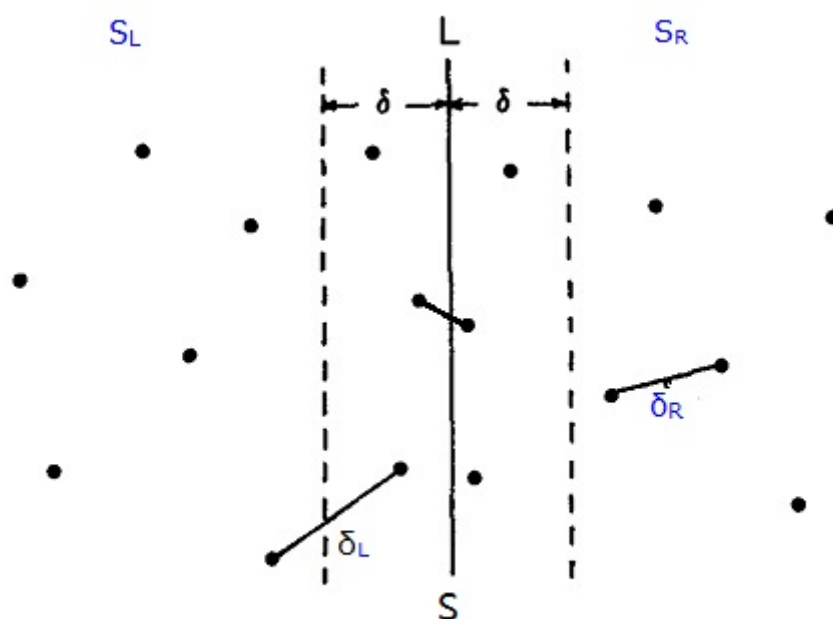
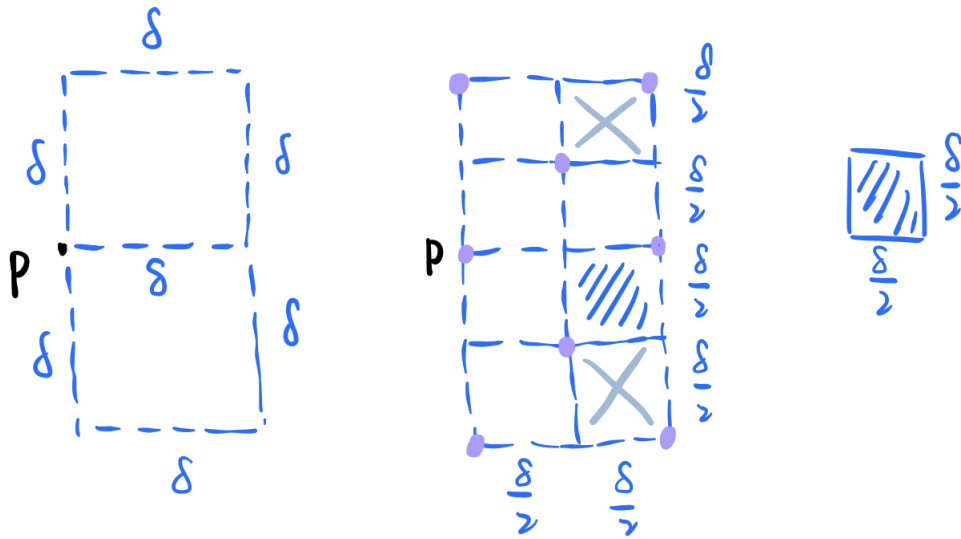


Figure 1.

进一步考虑位于  $S_L$  虚线框中的点  $p$ , 与其距离小于  $\delta$  的  $S_R$  中的点  $q$ , 一定位于下图蓝色框之中。



将蓝色方框划分为边长为  $\frac{\delta}{2}$  的小框，若有两个点位于同一个框中，则它们的曼哈顿距离小于  $\delta$ ，形成矛盾，以及上图中画叉的两个方块内的点与  $p$  的距离显然大于了  $\delta$  因此蓝色方框中的点最多只有 6 个。

(不过为了枚举方便还是取 8 个吧 QAQ，反正也差不多~)

在递归结束后将图1虚线框中的点提取出来，按  $y$  坐标排序，遍历  $S_L$  中符合要求的点，依次枚举其蓝色区域，计算曼哈顿距离。

## 参考代码

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include<math.h>
#include<stdbool.h>
#include<vector>
#include<algorithm>
#include<iostream>
#define x first
#define y second
using namespace std;
typedef long long ll;
typedef unsigned long long ull;
typedef pair<double,double> pi;
const int N=1000005;
const double inf=1e18;
int getint(){
    char c;int flag=0,num=0;
    while((c=getchar())<'0' || c>'9')if(c=='-')flag=1;
    while(c>='0'&&c<='9'){num=num*10+c-48;c=getchar();}
    return flag?-num:num;
}
pi a[N];
double dis(pi a,pi b){
    return fabs(a.x-b.x)+fabs(a.y-b.y);
}
bool cmpy(pi a,pi b){return a.y<b.y;}
double divi(int l,int r){
    if(l>=r) return inf;
    else if(l+1==r) return dis(a[l],a[r]);
    int mid=(l+r)>>1;
```

```

double d=divi(l,mid);
d=min(d,divi(mid+1,r));
double ans=d;

vector<pi> L,R;
for(int i=1;i<=mid;i++)
    if(a[mid].x-a[i].x<=d)
        L.push_back(a[i]);
for(int i=mid+1;i<=r;i++)
    if(a[i].x-a[mid].x<=d)
        R.push_back(a[i]);
sort(L.begin(),L.end(),cmpy);
sort(R.begin(),R.end(),cmpy);
int sz1=L.size(),sz2=R.size();
for(int i=0,j=0;i<sz1;i++){
    while(j<sz2 && R[j].y+d<L[i].y) j++;
    for(int k=j;k<sz2&&k<=j+8;k++)
        ans=min(ans,dis(L[i],R[k]));
}
L.clear(),R.clear();
return ans;
}

int main(){
    int n=getint();
    for(int i=1;i<=n;i++)
        scanf("%lf%lf",&a[i].x,&a[i].y);
    sort(a+1,a+n+1);
    printf("%.3f\n",divi(1,n));
    return 0;
}

```

以上实现方式时间复杂度为 $\Theta(n \lg^2 n)$ ，可以通过在分治的时候同时进行归并排序的方法使时间复杂度变为 $\Theta(n \lg n)$ ，实现如下：

```

#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include<math.h>
#include<stdbool.h>
#include<vector>
#include<algorithm>
#include<iostream>
#define x first
#define y second
using namespace std;
typedef long long ll;
typedef unsigned long long ull;
typedef pair<double,double> pi;
const int N=1000005;
const double inf=1e18;
int getint(){
    char c;int flag=0,num=0;
    while((c=getchar())<'0' || c>'9') if(c=='-') flag=1;
    while(c>='0'&&c<='9'){num=num*10+c-48;c=getchar();}
    return flag?-num:num;
}

pi a[N],tep[N];

```

```

double dis(pi a,pi b){
    return fabs(a.x-b.x)+fabs(a.y-b.y);
}
bool cmpy(pi a,pi b){return a.y<b.y;}
double divi(int l,int r){
    if(l>=r) return inf;
    else if(l+1==r) return dis(a[l],a[r]);
    int mid=(l+r)>>1;
    double d=divi(l,mid);
    d=min(d,divi(mid+1,r));
    double ans=d;

    vector<pi> L,R;
    for(int i=l;i<=mid;i++)
        if(a[mid].x-a[i].x<=d)
            L.push_back(a[i]);
    for(int i=mid+1;i<=r;i++)
        if(a[i].x-a[mid].x<=d)
            R.push_back(a[i]);
    int sz1=L.size(),sz2=R.size();
    for(int i=0,j=0;i<sz1;i++){
        while(j<sz2 && R[j].y+d<L[i].y) j++;
        for(int k=j;k<sz2&&k<=j+8;k++)
            ans=min(ans,dis(L[i],R[k]));
    }
    L.clear(),R.clear();
    for(int i=l,j=mid+1,k=1;i<=mid||j<=r;){
        if(i<=mid && (j>r || a[i].y<=a[j].y))
            tep[k++]=a[i++];
        else tep[k++]=a[j++];
    }
    return ans;
}
int main(){
    int n=getint();
    for(int i=1;i<=n;i++)
        scanf("%lf%lf",&a[i].x,&a[i].y);
    sort(a+1,a+n+1);
    printf("%.3f\n",divi(1,n));
    return 0;
}

```